



Unión Europea

Fondo Social Europeo

El FSE invierte en tu futuro

Laravel

Seeders y Fakers…

Pepe Devesa



GENERALITAT VALENCIANA

Conselleria d'Educació, Investigació, Cultura i Esport



GOBIERNO DE ESPAÑA

MINISTERIO DE EDUCACIÓN, CULTURA Y DEPORTE

Continguts

1 Los Seeders **3**
1.1 Añadiendo los seeders a la aplicación 4
1.2 Lanzar los seeders 5
1.3 Los fakers 5

En las pruebas que hemos hecho hasta ahora, para tener datos con que probar la aplicación, nos hemos limitado a añadirlos a mano desde phpMyAdmin, o bien desde el código con algunos datos simples como “Título de prueba 1” o cosas similares.

Dado que los datos de inicio son necesarios para probar algunas funcionalidades básicas de la aplicación, como son las búsquedas y filtrados, y dado que los formularios para dar de alta y gestionar estos datos normalmente no se tienen listos hasta etapas más tardías, puede resultar conveniente disponer de algún mecanismo que genere estos datos de prueba al inicio, sin preocuparnos de tocar la base de datos a mano o alterar el código de la aplicación para ello. En este aspecto, los seeders juegan un papel importante.

En las pruebas que hemos hecho hasta ahora, para tener datos con que probar la aplicación, nos hemos limitado a añadirlos a mano desde phpMyAdmin, o bien desde el código con algunos datos simples como “Título de prueba 1” o cosas similares.

Dado que los datos de inicio son necesarios para probar algunas funcionalidades básicas de la aplicación, como son las búsquedas y filtrados, y dado que los formularios para dar de alta y gestionar estos datos normalmente no se tienen listos hasta etapas más tardías, puede resultar conveniente disponer de algún mecanismo que genere estos datos de prueba al inicio, sin preocuparnos de tocar la base de datos a mano o alterar el código de la aplicación para ello. En este aspecto, los seeders y factories juegan un papel importante.

1 Los Seeders

Los seeders son clases especiales que permiten sembrar (seed) de contenido una aplicación. Para crearlos, utilizamos el comando `php artisan` como sigue:

```
php artisan make:seeder NombreSeeder
```

Esto creará una clase llamada NombreSeeder en la carpeta database/seeds (hasta Laravel 7) o `database/seeders` (desde Laravel 8). En el método `run` de dicha clase podemos crear los elementos que necesitemos añadir a la base de datos.

Por ejemplo, vamos a crear en nuestro proyecto biblioteca un seeder llamado `LibrosSeeder` para crear libros, y otro llamado AutoresSeeder para autores:

```
php artisan make:seeder LibrosSeeder
php artisan make:seeder AutoresSeeder
```

Editamos el método run del seeder que hemos creado (el de libros, por ejemplo), y definimos este código para crear un autor con un libro asociado (deberemos incorporar con `use` los modelos de `Autor` y `Libro` previamente):

```
public function run()
{
    $autor = new Autor();
    $autor->nombre = "Juan Seeder";
    $autor->nacimiento = 1960;
    $autor->save();
    $libro = new Libro();
    $libro->titulo = "El libro del Seeder";
    $libro->editorial = "Seeder S.A.";
    $libro->precio = 10;
    $libro->autor()->associate($autor);
    $libro->save();
}
```

De forma similar, podríamos editar el método run de AutoresSeeder para crear, en este caso, un autor (sin libro asociado, normalmente).

```
public function run()
{
    $autor = new Autor();
    $autor->nombre = "Autor Suelto";
    $autor->nacimiento = 1951;
    $autor->save();
}
```

1.1 Añadiendo los seeders a la aplicación

Por defecto, los seeders que creamos no forman parte de la aplicación aún, en el sentido de que aún no los podemos ejecutar. Para ello, debemos darlos de alta en el seeder general, llamado DatabaseSeeder, ubicado en la misma carpeta que los seeders que definimos:

```
class DatabaseSeeder extends Seeder
{
    public function run()
    {
        ...
        $this->call(AutoresSeeder::class);
        $this->call(LibrosSeeder::class);
    }
}
```



NOTA: es importante el orden en que colocamos los seeders dentro de este método, como veremos después, ya que esto indicará en qué orden se crean los datos de prueba en la aplicac

1.2 Lanzar los seeders

Si queremos ejecutar los seeders para que añadan los datos, emplearemos este comando:

```
php artisan db:seed
```

Esto lanzará todos los seeders que tengamos declarados en la clase DatabaseSeeder. Si sólo queremos lanzar uno en concreto, podemos hacer lo siguiente:

```
php artisan db:seed --class=LibrosSeeder
```

También puede ser necesario (y a veces conveniente) limpiar la base de datos y llenarla desde cero con los datos de los seeds para empezar a probar la aplicación. En este caso, el comando es el siguiente:

```
php artisan migrate:fresh --seed
```

1.3 Los fakers

Estaremos de acuerdo en que generar datos del tipo “Autor 1”, “Autor 2”, etc, no queda demasiado “real” en una aplicación, por mucho que sean datos de prueba. Por ello, Laravel nos proporciona **los fakers** para generar datos al azar con una apariencia determinada. Así, podemos generar nombres reales aleatorios, o direcciones de correo electrónico, o frases, o textos largos. Para generar estos datos de prueba, Laravel cuenta con una clase llamada `Faker`, cuyo objeto está automáticamente incorporado en el factory a través de la propiedad `$this->faker`.

Internamente dispone de métodos para generar datos de distintos tipos.Algunos de los más habituales son:

- `name`: genera un nombre de persona. Admite como parámetro opcional “male” o “female” para generar nombres masculinos o femeninos, respectivamente.
- `sentence`: genera una frase corta. Admite como parámetro opcional un número, indicando cuántas palabras generar.
- `word`: genera una palabra aleatoria.
- `text`: genera un texto largo.
- `phoneNumber`: genera un número de teléfono.
- `email`: genera un e-mail aleatorio.
- `randomNumber`: genera un número aleatorio. Como alternativa, también se tiene numberBetween, que genera un número aleatorio entre un mínimo y un máximo pasados como parámetro.

... etc (aquí podéis consultar más posibilidades al respecto).

y vamos a usar el componente Faker para crear un libro de forma automatica, el archivo final quedara de la siguiente forma, recuerden que para usar Faker es necesario importar la clase con la intruccion use:

```
...
use Faker\Factory as Faker;
...
class LibrosSeeder extends Seeder
{
/**
* Run the database seeds.
*/
    public function run(): void
    {
    //
            $faker = Faker::create();

        $libro = new Libro();
        $libro->titulo = 'El libro de ' . $faker->firstNameFemale;
        $libro->editorial = $faker->randomElement(['Alfguara','Palneta','
        Bromera']);
        $libro->precio = 10;
        $libro->autor_id=1;
        $libro->created_at= date('Y-m-d H:m:s'),
        $libro->updated_at = date('Y-m-d H:m:s')
        $libro->save();
    }
}
```

Con esto ya tendremos la estructura de la tabla y un seeder para poblar, con esto ahora usaremos el siguiente comando para crear la tabla en la BD y llenarla:

```
php artisan migrate --seed
```